Grupo 8

ImportTrack

Integrantes:

- María Alay
- John Carreño
- Sebastián Escobar
- Félix Lucas

Manual de Operaciones CRUD para la Tabla Cliente.

Clase CConexion – Conexión a la Base de Datos

Este módulo define la clase CConexion, encargada de establecer la conexión con la base de datos MySQL utilizada por el sistema.

Descripción del método ConexionBaseDeDatos:

- Intenta establecer una conexión con el servidor MySQL utilizando las credenciales y parámetros especificados:
  - Usuario
  - Contraseña
  - Host
  - Base de datos
  - Puerto
- Si la conexión es exitosa, muestra el mensaje "Conexión Correcta" en la consola y retorna el objeto de conexión.
- Si ocurre un error, captura la excepción, muestra un mensaje indicando el problema y retorna la variable conexion.

Uso:

Este método se invoca para obtener una conexión activa que permita ejecutar consultas SQL sobre la base de datos definida.

```python
import mysql.connector

class CConexion:
    def ConexionBaseDeDatos():
        try:
            conexion = mysql.connector.connect(user='root',password='johnc1to',
                                               host='127.0.0.1',
                                               database='importtrack',
                                               port='3306')
            print("Conexion Correcta")
            return conexion
        except mysql.connector.Error as error:
            print("Error al conectarse a la base de datos {}".format(error))
            return conexion
    ConexionBaseDeDatos()
```

Clase CClientes – Operaciones CRUD sobre la tabla Cliente

Esta clase contiene los métodos necesarios para realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre la tabla Cliente de la base de datos.
Cada método establece su propia conexión utilizando la clase CConexion, ejecuta la consulta SQL correspondiente y luego cierra la conexión.

1. mostrarClientes() – Lectura de todos los clientes

- Función: Recupera todos los registros existentes en la tabla Cliente.
- Procedimiento:
    1. Establece conexión con la base de datos.
    2. Ejecuta select * from Cliente;.
    3. Retorna todos los resultados obtenidos.

```python
def mostrarClientes():
    try:
        cone = CConexion.ConexionBaseDeDatos()
        cursor = cone.cursor()
        cursor.execute("select * from Cliente;")
        miResultado = cursor.fetchall()
        cone.commit()
        cone.close()
        return miResultado
    except mysql.connector.Error as error:
        print("Error de mostrar datos {}".format(error))
```

2. ingresarClientes(direccion, correo, tipo) – Inserción de un nuevo cliente

- Función: Inserta un nuevo registro en la tabla Cliente.
- Parámetros:
    - direccion (str): Dirección del cliente.
    - correo (str): Correo electrónico del cliente.
    - tipo (str): Tipo de cliente.
- Procedimiento:

1. Ejecuta insert into Cliente (Direccion, Correo, Tipo) values (%s, %s, %s);.
2. Confirma la inserción con commit().

```python
def ingresarClientes(direccion,correo,tipo):
    try:
        cone = CConexion.ConexionBaseDeDatos()
        cursor = cone.cursor()
        sql = "insert into Cliente(Direccion,Correo,Tipo) values (%s,%s,%s);"
        valores = (direccion,correo,tipo)
        cursor.execute(sql,valores)
        cone.commit()
        print(cursor.rowcount,"Registro Ingresado")
        cone.close()
    except mysql.connector.Error as error:
        print("Error de ingreso de datos {}".format(error))
```

3. modificarClientes(id, direccion, correo, tipo) – Actualización de datos de un cliente

- Función: Modifica un registro existente en la tabla Cliente.
- Parámetros:
  - id (int): ID del cliente a modificar.
  - direccion, correo, tipo: Nuevos valores para los campos.
- Procedimiento:

1. Ejecuta update Cliente set Direccion=%s, Correo=%s, Tipo=%s where ID_Cliente=%s;.
2. Confirma los cambios con commit().

```python
def modificarClientes(id,direccion,correo,tipo):
    try:
        cone = CConexion.ConexionBaseDeDatos()
        cursor = cone.cursor()
        sql = "update Cliente set Cliente.Direccion=%s, Cliente.Correo=%s, Cliente.Tipo=%s where ID_Cliente=%s;"
        valores = (direccion,correo,tipo,id)
        cursor.execute(sql,valores)
        cone.commit()
        print(cursor.rowcount,"Registro Actualizado")
        cone.close()
    except mysql.connector.Error as error:
        print("Error de actualizacion de datos {}".format(error))
```

4. eliminarClientes(id) – Eliminación de un cliente

- Función: Elimina un registro de la tabla Cliente según su ID.
- Parámetros:
  - id (int): ID del cliente a eliminar.
- Procedimiento:

1. Ejecuta delete from Cliente where ID_Cliente=%s;.
2. Confirma la eliminación con commit().

```python
def eliminarClientes(id):
    try:
        cone = CConexion.ConexionBaseDeDatos()
        cursor = cone.cursor()
        sql = "delete from Cliente where Cliente.ID_Cliente=%s"
        valores = (id,)
        cursor.execute(sql,valores)
        cone.commit()
        print(cursor.rowcount,"Registro eliminado")
        cone.close()
    except mysql.connector.Error as error:
        print("Error de eliminacion de datos {}".format(error))
```

5. seleccionarCliente(id) – Consulta de un cliente específico

- Función: Recupera los datos de un cliente según su ID.
- Parámetros:
  - id (int): ID del cliente a consultar.

- Procedimiento:

1. Ejecuta select * from Cliente where ID_Cliente=%s;.
2. Retorna los datos del cliente encontrado.

```python
def seleccionarCliente(id):
    try:
        cone = CConexion.ConexionBaseDeDatos()
        cursor = cone.cursor()
        sql = "select * from Cliente where Cliente.ID_Cliente=%s"
        valores = (id,)
        cursor.execute(sql,valores)
        miResultado = cursor.fetchall()
        cone.commit()
        print(cursor.rowcount,"Mostrando al cliente seleccionado")
        cone.close()
        return miResultado
    except mysql.connector.Error as error:
        print("Error de seleccion de cliente {}".format(error))
```

Clase FormularioClientes – Interfaz Gráfica para Gestión de Clientes

Este módulo implementa la interfaz gráfica del sistema **ImportTrack** utilizando **Tkinter**. Permite al usuario interactuar con la tabla Cliente de la base de datos mediante operaciones CRUD a través de formularios y tablas visuales.

Funciones y Componentes Principales

1. Formulario() – Inicialización de la ventana principal

- **Función:** Crea y configura la ventana principal de la aplicación, incluyendo tamaños, título e icono.
- **Componentes:**
  - **LabelFrame Datos del Cliente:** Contiene los campos para ingresar o modificar datos de un cliente.
    - Entradas: ID, Direccion, Correo
    - ComboBox: Tipo (opciones: Natural, Jurídica)
    - Botones: Guardar, Modificar, Eliminar
  - **Logo del sistema:** Imagen redimensionada y mostrada en la ventana.
  - **LabelFrame Lista de Clientes:** Tabla (Treeview) que muestra todos los registros de clientes.
    - Columnas: ID_Cliente, Direccion, Correo, Tipo
    - Permite seleccionar un registro para cargarlo en los campos de entrada.
  - **LabelFrame Buscar Cliente:** Permite filtrar la tabla por ID.
    - Entrada: ID del cliente a buscar
    - Botones: Buscar, Mostrar Todos

```python
def Formulario():
    global base
    global textBoxID
    global textBoxDireccion
    global textBoxCorreo
    global combo
    global groupBox
    global tree
    global textBoxSeleccion
    try:
        base = Tk()
        base.geometry("1210x480")
        base.title("ImportTrack")
        imagen_original = Image.open("Logo.png")
        imagen_redimensionada = imagen_original.resize((32, 32))
        icono = ImageTk.PhotoImage(imagen_redimensionada)
        base.iconphoto(False, icono)

        groupBox = LabelFrame(base,text="Datos del Cliente",padx=5,pady=5)
        groupBox.grid(row=0,column=1,padx=10,pady=10)

        labelId = Label(groupBox,text="ID: ",width=13,font=("arial",12)).grid(row=0,column=0)
        textBoxID = Entry(groupBox)
        textBoxID.grid(row=0,column=1)

        labelDireccion = Label(groupBox,text="Direccion: ",width=13,font=("arial",12)).grid(row=1,column=0)
        textBoxDireccion = Entry(groupBox)
        textBoxDireccion.grid(row=1,column=1)

        labelCorreo = Label(groupBox,text="Correo: ",width=13,font=("arial",12)).grid(row=2,column=0)
        textBoxCorreo = Entry(groupBox)
```

```python
        textBoxCorreo.grid(row=2,column=1)

        labelTipo = Label(groupBox,text="Tipo: ",width=13,font=("arial",12)).grid(row=3,column=0)
        seleccionTipo = tk.StringVar()
        combo = ttk.Combobox(groupBox,values=["Natural","Juridica"],textvariable=seleccionTipo)
        combo.grid(row=3,column=1)
        seleccionTipo.set("Natural")

        Button(groupBox,text="Guardar",width=10,command=guardarRegistros).grid(row=4,column=0)
        Button(groupBox,text="Modificar",width=10,command=modificarRegistros).grid(row=4,column=1)
        Button(groupBox,text="Eliminar",width=10,command=eliminarRegistros).grid(row=4,column=2)


        imagen_logo = Image.open("ImportTrackLogo.png")
        imagen_logo = imagen_logo.resize((300, 300))
        imagen_tk = ImageTk.PhotoImage(imagen_logo)

        labelLogo = Label(base, image=imagen_tk)
        labelLogo.image = imagen_tk
        labelLogo.grid(row=1, column=1, pady=(0, 20))


        groupBox = LabelFrame(base,text="Lista de Clientes",padx=5,pady=5)
        groupBox.grid(row=1,column=0,padx=5,pady=5)

        tree = ttk.Treeview(groupBox,columns=("ID_Cliente","Direccion","Correo","Tipo"),show='headings',height=1
        tree.column("# 1",anchor=CENTER)
        tree.heading("# 1",text="ID_Cliente")
        tree.column("# 2",anchor=CENTER)
        tree.heading("# 2",text="Direccion")
        tree.column("# 3",anchor=CENTER)
```

```
    tree.heading("# 3",text="Correo")
    tree.column("# 4",anchor=CENTER)
    tree.heading("# 4",text="Tipo")

    for row in CClientes.mostrarClientes():
        tree.insert("","end",values=row)

    tree.bind("<<TreeviewSelect>>",seleccionarRegistro)

    tree.pack()

    groupBoxBusqueda = LabelFrame(base, text="Buscar Cliente", padx=5, pady=5)
    groupBoxBusqueda.grid(row=0, column=0, padx=5, pady=5)

    labelSeleccion = Label(groupBoxBusqueda, text="Ingrese un ID para buscar:", font=("arial",12))
    labelSeleccion.grid(row=0, column=0, sticky="w")
    textBoxSeleccion = Entry(groupBoxBusqueda)
    textBoxSeleccion.grid(row=1, column=0, padx=5, pady=5)

    Button(groupBoxBusqueda, text="Buscar",command=seleccionarCliente).grid(row=2, column=0, padx=5, pady=5)
    Button(groupBoxBusqueda, text="Mostrar Todos", command=actualizarTreeView).grid(row=3, column=0, padx=5,




    base.mainloop()
except ValueError as error:
    print("Error al mostrar la interfaz, error: {}".format(error))
```

2. guardarRegistros() – Guardar un nuevo cliente

- Recupera los datos de los campos Direccion, Correo y Tipo.
- Invoca CClientes.ingresarClientes() para insertar el registro en la base de datos.
- Muestra un mensaje de confirmación y actualiza la tabla.

```
def guardarRegistros():
    global textBoxDireccion,textBoxCorreo,combo,groupBox

    try:
        if textBoxDireccion is None or textBoxCorreo is None or combo is None:
            print("Los Widget no estan inicializados")
            return
        direccion = textBoxDireccion.get()
        correo = textBoxCorreo.get()
        tipo = combo.get()
        CClientes.ingresarClientes(direccion,correo,tipo)
        messagebox.showinfo("Informacion","Los datos fueron guardados")
        actualizarTreeView()
        textBoxDireccion.delete(0,END)
        textBoxCorreo.delete(0,END)
    except ValueError as error:
        print("Error al mostrar los datos {}".format(error))
```

3. actualizarTreeView() – Refrescar la tabla de clientes

- Limpia el contenido actual del Treeview y lo rellena con los registros actualizados de la base de datos.

```
def actualizarTreeView():
    global tree
    try:
        tree.delete(*tree.get_children())
        datos = CClientes.mostrarClientes()
        for row in CClientes.mostrarClientes():
            tree.insert("","end",values=row)
    except ValueError as error:
        print("Error al actulizar tabla {}".format(error))
```

4. seleccionarRegistro(evento) – Selección de un registro en la tabla

- Cuando se selecciona un registro en el Treeview, carga sus valores en los campos de entrada para poder modificar o eliminar.

```
def seleccionarRegistro(evento):
    try:
        itemSeleccionado = tree.focus()
        if itemSeleccionado:
            values = tree.item(itemSeleccionado)['values']
            textBoxID.delete(0,END)
            textBoxID.insert(0,values[0])
            textBoxDireccion.delete(0,END)
            textBoxDireccion.insert(0,values[1])
            textBoxCorreo.delete(0,END)
            textBoxCorreo.insert(0,values[2])
            combo.set(values[3])
    except ValueError as error:
        print("Error al seleccionar registro {}".format(error))
```

5. modificarRegistros() – Actualizar un cliente existente

- Recupera los datos de los campos de entrada y llama a CClientes.modificarClientes() para actualizar la base de datos.
- Muestra un mensaje de confirmación y actualiza la tabla.

```python
def modificarRegistros():
    global textBoxID,textBoxDireccion,textBoxCorreo,combo,groupBox
    try:
        if textBoxID is None or textBoxDireccion is None or textBoxCorreo is None or combo is None:
            print("Los Widget no estan inicializados")
            return
        id = textBoxID.get()
        direccion = textBoxDireccion.get()
        correo = textBoxCorreo.get()
        tipo = combo.get()
        CClientes.modificarClientes(id,direccion,correo,tipo)
        messagebox.showinfo("Informacion","Los datos fueron actualizados")
        actualizarTreeView()
        textBoxID.delete(0,END)
        textBoxDireccion.delete(0,END)
        textBoxCorreo.delete(0,END)
    except ValueError as error:
        print("Error al actualizar los datos {}".format(error))
```

6. eliminarRegistros() – Eliminar un cliente

- Obtiene el ID del cliente a eliminar y llama a CClientes.eliminarClientes().
- Muestra un mensaje de confirmación y actualiza la tabla.

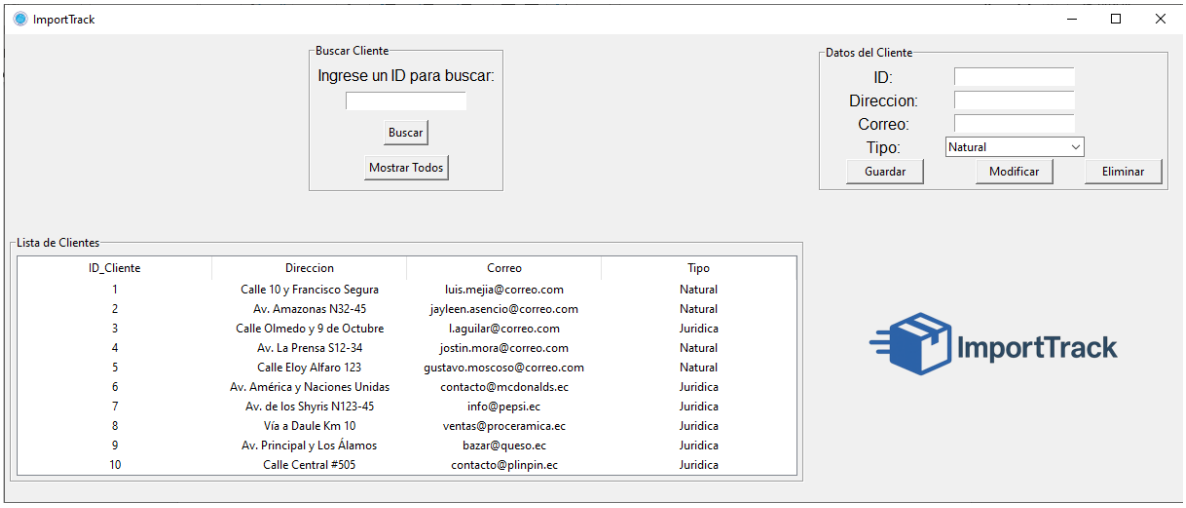
```python
def eliminarRegistros():
    global textBoxID,textBoxDireccion,textBoxCorreo,combo,groupBox
    try:
        if textBoxID is None:
            print("El Widget no esta inicializados")
            return
        id = textBoxID.get()
        CClientes.eliminarClientes(id)
        messagebox.showinfo("Informacion","El registro fue eliminado")
        actualizarTreeView()
        textBoxCorreo.delete(0,END)
        textBoxDireccion.delete(0,END)
        textBoxID.delete(0,END)
    except ValueError as error:
        print("Error al eliminar los datos {}".format(error))
```

7. seleccionarCliente() – Buscar un cliente específico

- Obtiene el ID ingresado por el usuario, consulta la base de datos usando CClientes.seleccionarCliente() y muestra únicamente ese registro en la tabla.

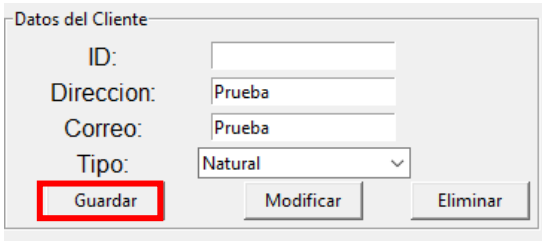
```python
def seleccionarCliente():
    global textBoxSeleccion
    try:
        if textBoxSeleccion is None:
            print("No se ha ingresado una ID")
            return
        id = textBoxSeleccion.get()
        tree.delete(*tree.get_children())
        datos = CClientes.seleccionarCliente(id)
        for row in datos:
            tree.insert("","end",values=row)
        textBoxSeleccion.delete(0,END)
    except ValueError as error:
        print("Error al actualizar los datos {}".format(error))
```

Interfaz Gráfica Final



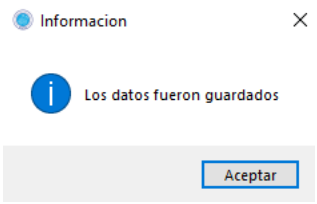
Uso:

Agregar un cliente



Para agregar un cliente basta con ingresar los campos de Direccion y Correo, para luego presionar el botón de guardar. Esto hará que la tabla se refresque y muestre el nuevo cliente que se agregó.

En caso de éxito se muestra el siguiente mensaje:



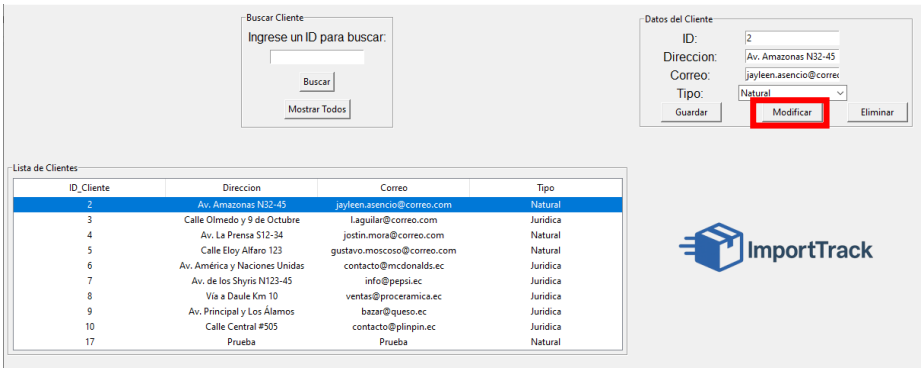
Y en consola se muestra lo siguiente:

```
1 Registro Ingresado
```

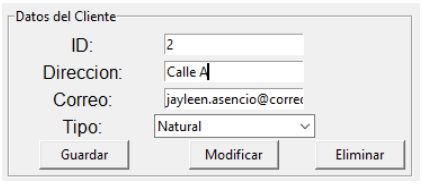
El ID no hace falta especificarlo dado que durante la creación de la tabla Clientes en el DML se lo definió como auto incremental.

Modificar un cliente

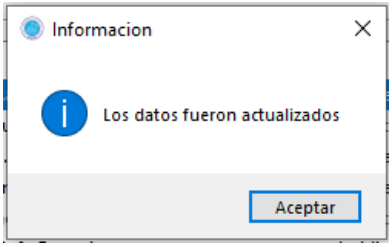
En la tabla podemos seleccionar el registro que queremos modificar, al hacer click sus datos se pasan al editor de datos.



Ahora podemos modificar su dirección, correo y tipo en el editor de datos



Luego presionamos el botón de modificar para confirmar la modificación. En caso de éxito se muestra el siguiente mensaje:

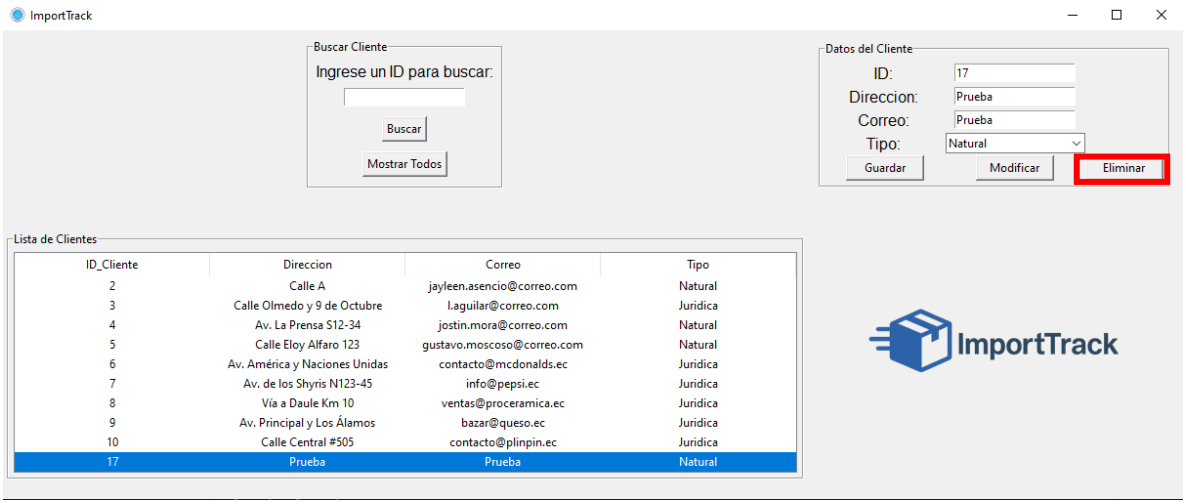


Y en consola se muestra lo siguiente:

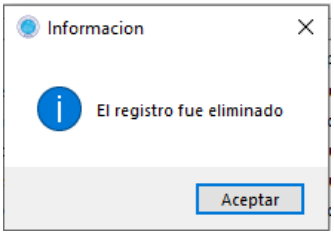
```
1 Registro Actualizado
```

Eliminar un cliente

En la tabla podemos seleccionar el registro que queremos eliminar, al hacer click sus datos se pasan al editor de datos.



Luego presionamos el botón de eliminar. En caso de éxito se muestra el siguiente mensaje:



Y en consola se muestra lo siguiente:

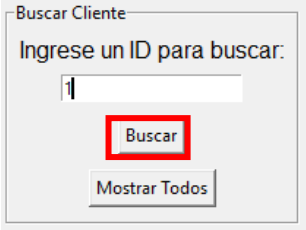
```
1 Registro eliminado
```

Para esta operación debemos tener cuidado, puesto que si intentamos eliminar un cliente que en nuestra base de datos ya tiene un pedido asociado no podremos por restricciones de la clave foránea, en ese caso se muestra el siguiente mensaje en consola:

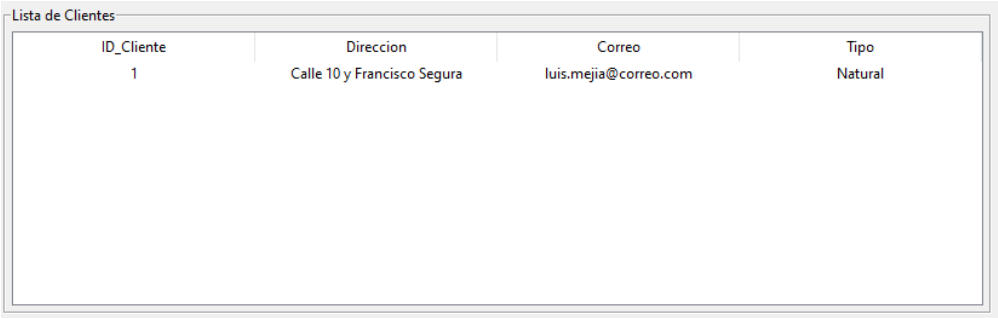
```
Error de eliminacion de datos 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`importtrack`.`pedido`, CONSTRAINT `fk_cliente_pedido` FOREIGN KEY (`ID_Cliente`) REFERENCES `cliente` (`ID_Cliente`))
```

Seleccionar un cliente

Para seleccionar un cliente solo ingresamos su ID en la respectiva casilla.



Luego presionamos el botón de buscar y la tabla se actualizará para solo mostrar el cliente con ese ID.



Si queremos volver a ver todos, presionamos el botón de mostrar todos.

